

INF473A

Algorithms and Competitive Programming

Lecture 6: (some) Graph algorithms !

Elias TSIGARIDAS



École Polytechnique

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

Shortest path(s)



What is a graph?

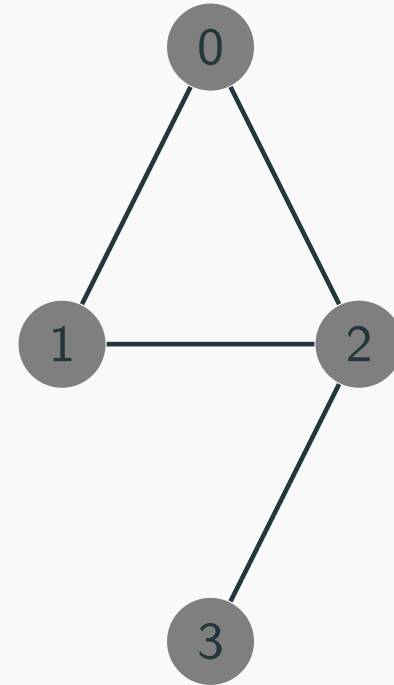
- Vertices

- Road intersections
- Computers
- Floors in a house
- Objects

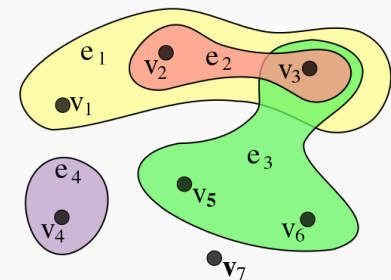
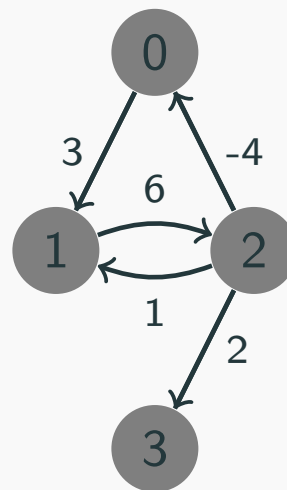
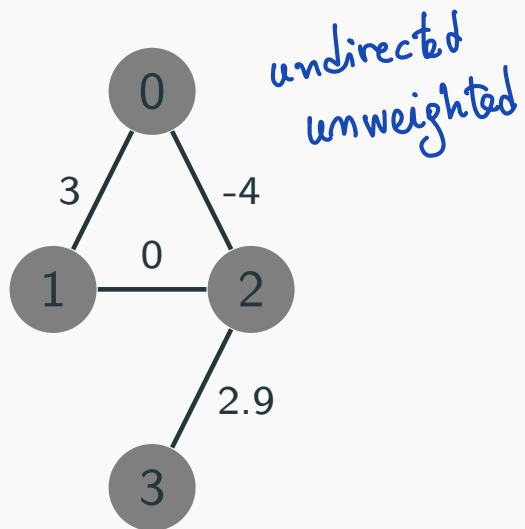
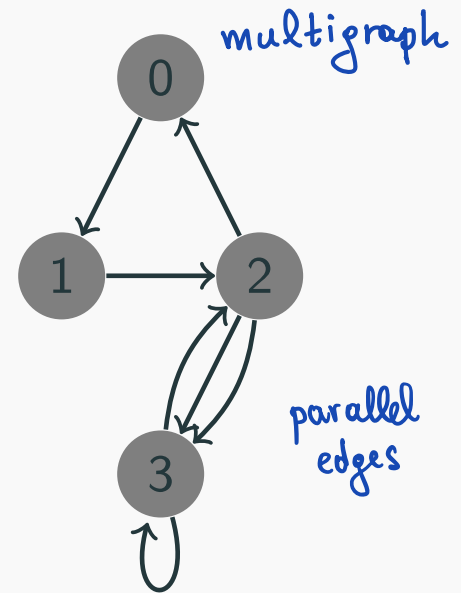
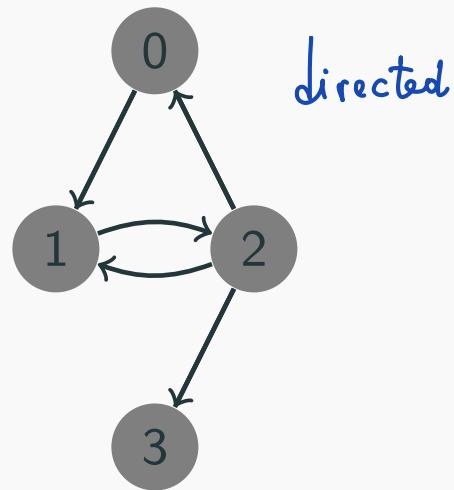
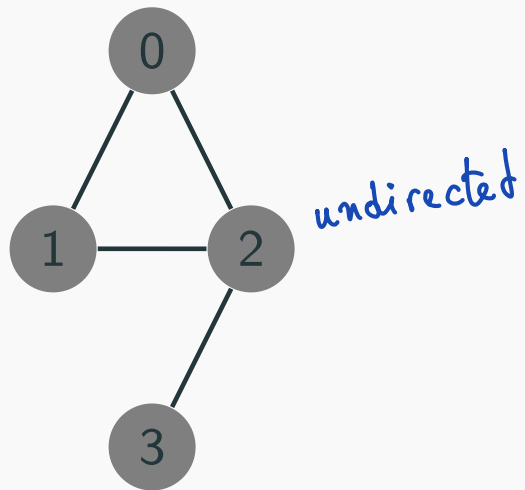
- Edges

- Roads
- Ethernet cables
- Stairs or elevators
- Relation between objects

set of vertices $V = \{v_1, \dots, v_n\}$ $|V| = n$
Edges $E \subseteq V \times V$ $|E| = m$
 $\{e_1, \dots, e_m\}$
 $e_i = [v_{i_1}, v_{i_2}]$



(many) Types of graphs



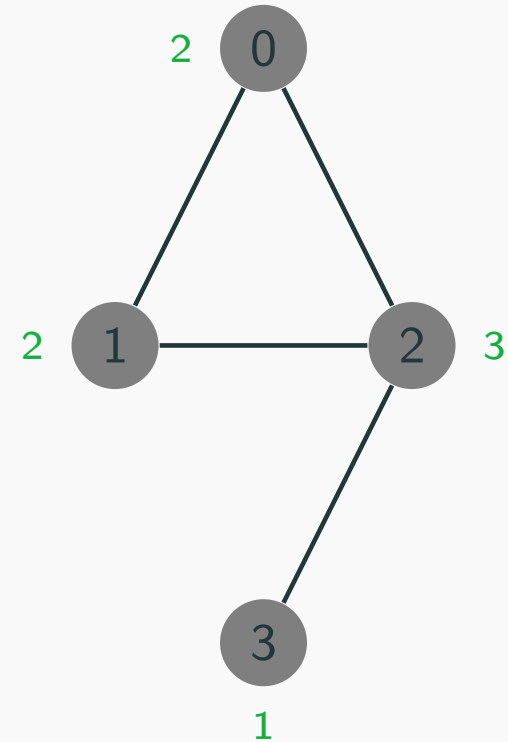
hypergraph
where an edge connects
many vertices

Vertex properties (undirected graph)

- Degree of a vertex
 - Number of adjacent edges
 - Number of adjacent vertices
- Handshaking lemma

$$\sum_{v \in V} \deg(v) = 2|E|$$

we count every edge twice



Representation

0: 1, 2
1: 0, 2
2: 0, 1, 3
3: 2

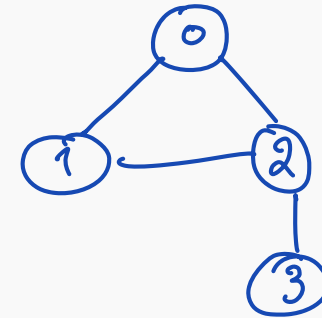
```
adj[0].size() // 2  
adj[1].size() // 2  
adj[2].size() // 3  
adj[3].size() // 1
```

directed graph
OUT-DEG IN-DEG

Adjacency Matrix

the straightforward way

Representation of a graph



- An easy way to store connectivity information
 - Checking if two nodes are directly connected: $O(1)$ time
- Make an $n \times n$ matrix A
 - $a_{ij} = 1$ if there is an edge from i to j
 - $a_{ij} = 0$ otherwise
- Uses $\Theta(n^2)$ memory
 - Only use when n is less than a few thousands,
 - and when the graph is dense

	0	1	2	3
0		1	1	
1	1		1	
2	1	1		1
3			1	

sparse [weights are 0/1]

undirected $\rightarrow$ symmetric matrix

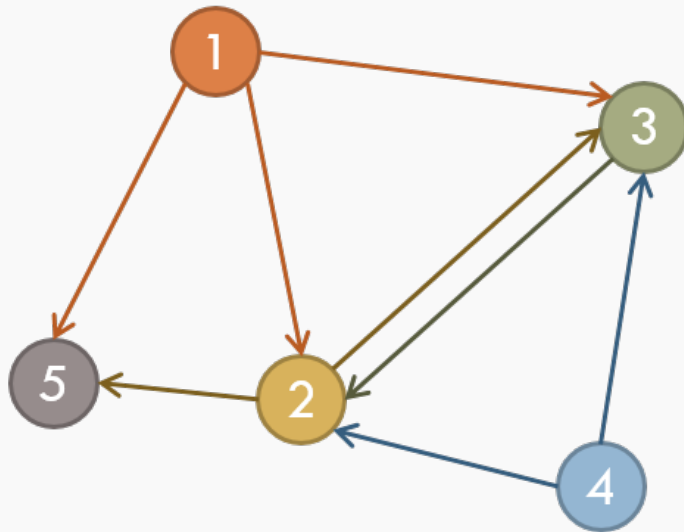
Important for storage & operations
eg. matrix-vector multiplication

Adjacency List [Use linked list for each node]

- Each node has a list of outgoing edges from it
 - Easy to iterate over edges incident to a certain node
 - The lists have variable lengths
 - Space usage: $\Theta(n + m)$

uses Linked Lists

(out)
length = degree of the vertex



From	To		
1	2	3	5
2	3	5	
3	2		
4	2	5	
5			

Q: Is the edge (i, j) present?

Complexity: $O(\text{DEG}(i))$
 $O(\text{OUT-DEG}(i))$

Adjacency list (undirected)

0: 1, 2

1: 0, 2

2: 0, 1, 3

3: 2

array of vectors

```
vector<int> adj[4];
```

```
adj[0].push_back(1);
```

```
adj[0].push_back(2);
```

```
adj[1].push_back(0);
```

```
adj[1].push_back(2);
```

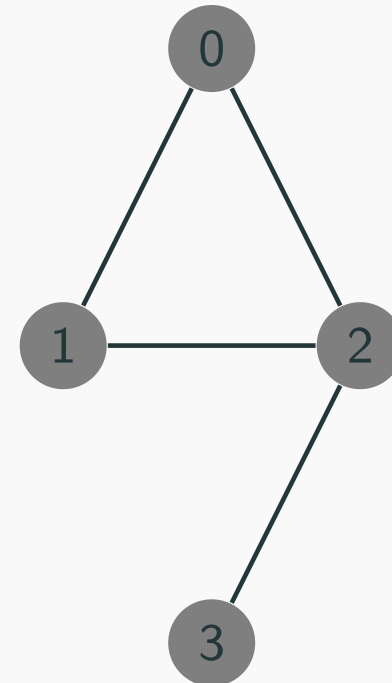
```
adj[2].push_back(0);
```

```
adj[2].push_back(1);
```

```
adj[2].push_back(3);
```

```
adj[3].push_back(2);
```

push_back 2 times
because it is undirected



Adjacency list (directed)

0: 1
1: 2
2: 0, 1, 3
3:

```
vector<int> adj[4];
```

```
adj[0].push_back(1);
```

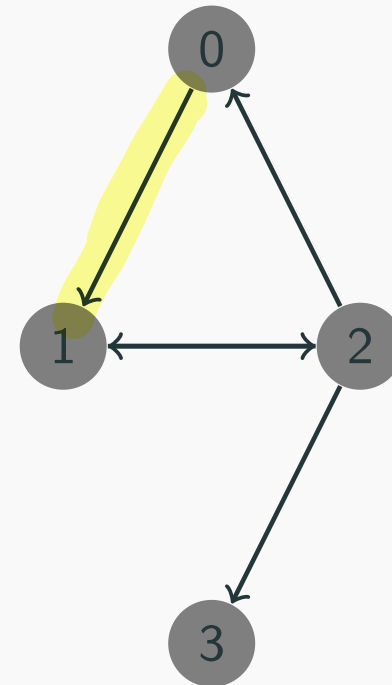
```
adj[1].push_back(2);
```

```
adj[2].push_back(0);
```

```
adj[2].push_back(1);
```

```
adj[2].push_back(3);
```

push-back
1 time



Notice the code is the same

How to represent the adjacency list?

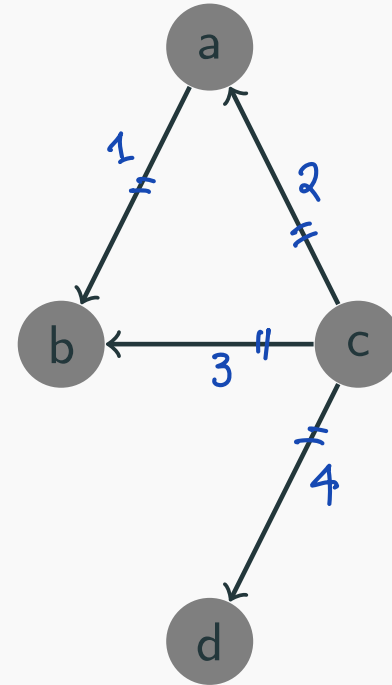
convert the Adjacency list to an array

- Linked List *slow*
- `vector<int> adj[4];`
- array

2 arrays E & H
and the out-degree

easy, relatively fast
no memory issues (compact)
BUT you have to initialize it
for every vertex
You can use it up to a
few thousands of vertices/edges

$$|V| = n$$
$$|E| = m$$



E (edges)

edge ID	To	NEXT
1	b	
2	a	
3	b	2
4	d	3

edge from
the same vertex

You also have to store
the (out)-Degree of the vertices

a	b	c	d
1		2 3 4	

$n = \# \text{ vertices}$

H (Head)
the first edge
in the linked
list

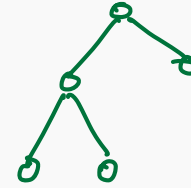
How to iterate to
all edges starting
from c

$m = \# \text{ edges}$

Special graphs

- Tree

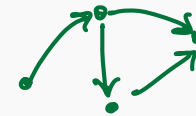
- A connected acyclic graph
- Most important type of special graphs
- Many problems are easier to solve on trees
- Alternate equivalent definitions:
 - A connected graph with $n - 1$ edges
 - An acyclic graph with $n - 1$ edges
 - There is exactly one path between every pair of nodes
 - An acyclic graph but adding any edge results in a cycle
 - A connected graph but removing any edge disconnects it



How do you recognize the special graphs?

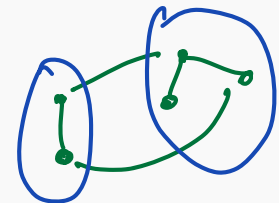
- Directed Acyclic Graph (DAG)

- Equivalent to a partial ordering of nodes



- Bipartite Graph:

There are two groups of nodes and no edge between the groups.



vertices
in the same

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

Shortest path(s)



Graph traversal

Graph Traversal or Graph Search

Visit all the nodes of a graph.

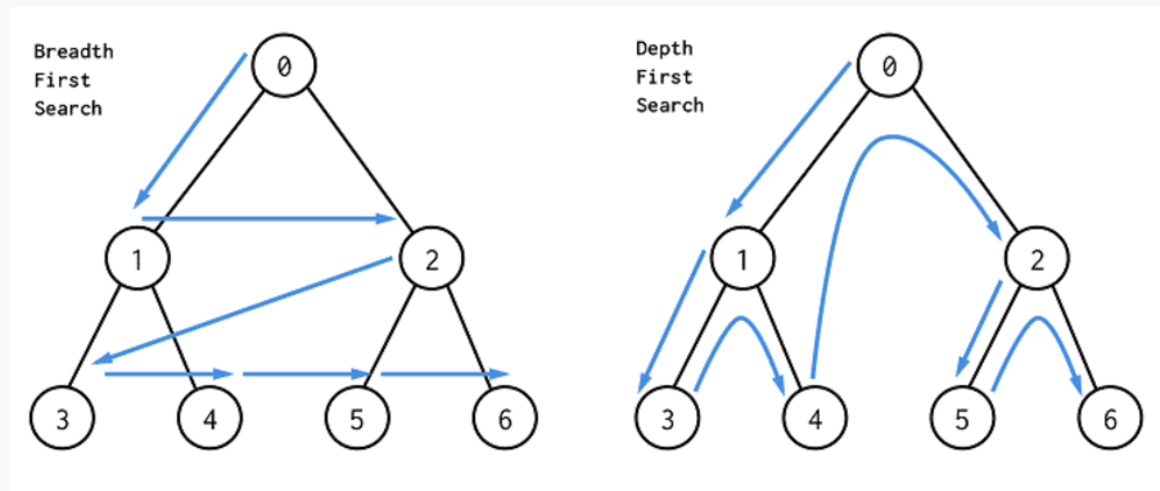
There are two main algorithms (differ on the order of traversal)

- **Breadth-First Search**

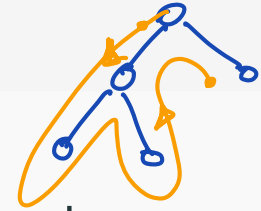
Traverse the graph connected to an adjacent node before traversing the next adjacent node.

- **Depth-First Search**

Explore (traverse) as far as possible along each branch before backtracking.



Depth-First Search



DFS(v): visits all the nodes reachable from v in depth-first order

- Mark v as visited
- For each edge $v \rightarrow u$:
 - If u is not visited, call **DFS**(u)
- Use non-recursive version if the recursion depth is too big (over a few thousands)
 - Replace recursive calls with a stack
- Complexity:
 - Time: $\mathcal{O}(|V| + |E|)$ or $\mathcal{O}(|V|^2)$
 - Space: $\mathcal{O}(|V|)$ (for the stack).

```
vector<int> adj[1000];    recursive version
vector<bool> visited(1000, false);
```

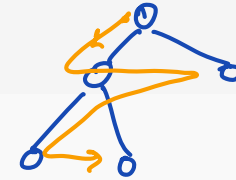
```
void dfs(int u) {
    if (visited[u]) {
        return;
    }

    visited[u] = true;

    for (int i = 0; i < adj[u].size(); i++) {
        int v = adj[u][i];
        dfs(v);
    }
}
```

for dense graphs
 $|E| \approx |V|^2$

Breadth-First Search



BFS(v): visits all the nodes reachable from v in breadth-first order

- Initialize a queue Q
- Mark v as visited and push it to Q
- While Q is not empty:
 - Take the front element of Q and call it w
 - For each edge $w \rightarrow u$:
 - If u is not visited, mark it as visited and push it to Q
- Complexity:
 - Time: $\mathcal{O}(|V| + |E|)$ or $\mathcal{O}(|V|^2)$
 - Space: $\mathcal{O}(|V|)$.

```
vector<int> adj[1000];  
vector<bool> visited(1000, false);
```

```
queue<int> Q;  
Q.push(start);  
visited[start] = true;
```

```
while (!Q.empty()) {  
    int u = Q.front(); Q.pop();
```

```
    for (int i = 0; i < adj[u].size(); i++) {  
        int v = adj[u][i];  
        if (!visited[v]) {  
            Q.push(v);  
            visited[v] = true;  
        }  
    }  
}
```

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

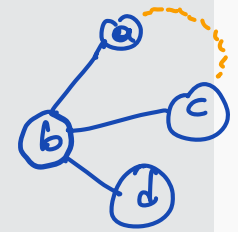
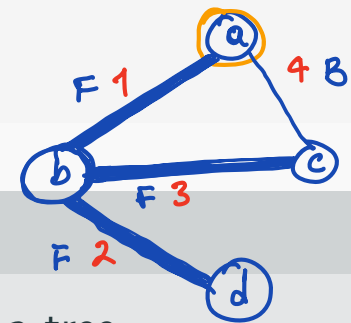
Shortest path(s)



Depth-first search tree

Constructing the tree from DFS

- When we do a DFS from a certain vertex, the edges we visit form a tree.
- When we go from vertex v_1 to vertex v_2 that have not visited before, then we call the edge *forward edge*.
- When we go from a vertex v_1 to a vertex v_2 that we have already visited, then we call the edge *backward edge*.
- The forward edges form a tree !



Applications

- A backward edge represents a cycle in the original graph.
- If there are no backward edges, then there are no cycles in the original graph (i.e. the graph is acyclic)

Analyzing the DFS tree

keep a diary while
we traverse the graph
and construct the tree

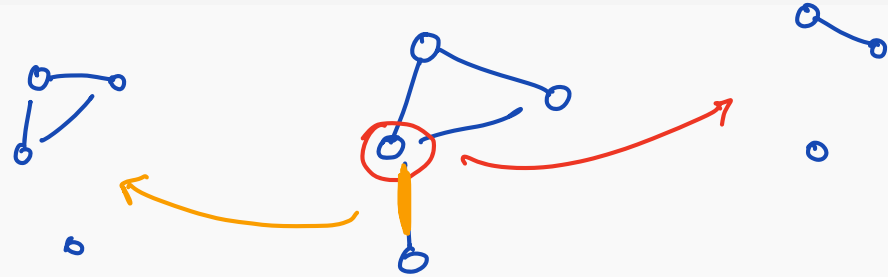
- Number the vertices in the order that we visit them in DFS.
- For each vertex, keep the smallest number of a vertex that we reach when exploring the subtree rooted at the current vertex.
- Complexity: $\mathcal{O}(|V| + |E|)$.

```
const int n = 1000;
vector<int> adj[n];
vector<int> low(n), num(n, -1);
int curnum = 0;

void analyze(int u, int p) {
    low[u] = num[u] = curnum++;
    for (int i = 0; i < adj[u].size(); i++) {
        int v = adj[u][i];
        if (v == p) continue;
        if (num[v] == -1) {
            analyze(v, u);
            low[u] = min(low[u], low[v]);
        } else {
            low[u] = min(low[u], num[v]);
        }
    }
}

for (int u = 0; u < n; u++) {
    if (num[u] == -1) {
        analyze(u, -1);
    }
}
```

Bridges + articulation points



- We have a **connected undirected** graph
- Find an edge, so that if you remove that edge, then the graph is no longer connected

Straightforward algorithm

Remove edges, one at a time, and find the connected components of the resulting graph.

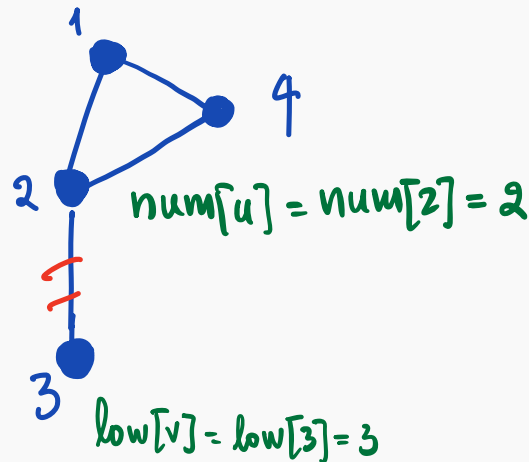
Complexity $\mathcal{O}(|E|(|V| + |E|)) = \mathcal{O}(m^2 + n)$.

Bridges

Use the DFS tree (and its analysis)

- A forward edge (u, v) is a bridge if and only if $\text{low}[v] > \text{num}[u]$
- Complexity $\mathcal{O}(n + m)$

if this is not the case, then
 $\text{low}[v] < \text{num}[u]$
it means that we can reach
u continuing from v



num = the order that we visit
a vertex using DFS

low = the smallest number of a vertex
that we reached while exploring
a subtree from this vertex

```
const int n = 1000;  
vector<int> adj[n];  
vector<int> low(n), num(n, -1);  
int curnum = 0;
```

```
vector<pair<int, int> > bridges;
```

```
void find_bridges(int u, int p) {  
    low[u] = num[u] = curnum++;  
    for (int i = 0; i < adj[u].size(); i++) {  
        int v = adj[u][i];  
        if (v == p) continue;  
        if (num[v] == -1) {  
            find_bridges(v, u);  
            low[u] = min(low[u], low[v]);  
        } else {  
            low[u] = min(low[u], num[v]);  
        }  
  
        if (low[v] > num[u]) {  
            bridges.push_back(make_pair(u, v));  
        }  
    }  
}  
  
for (int u = 0; u < n; u++) {  
    if (num[u] == -1) {  
        find_bridges(u, -1);  
    }  
}
```

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

Shortest path(s)

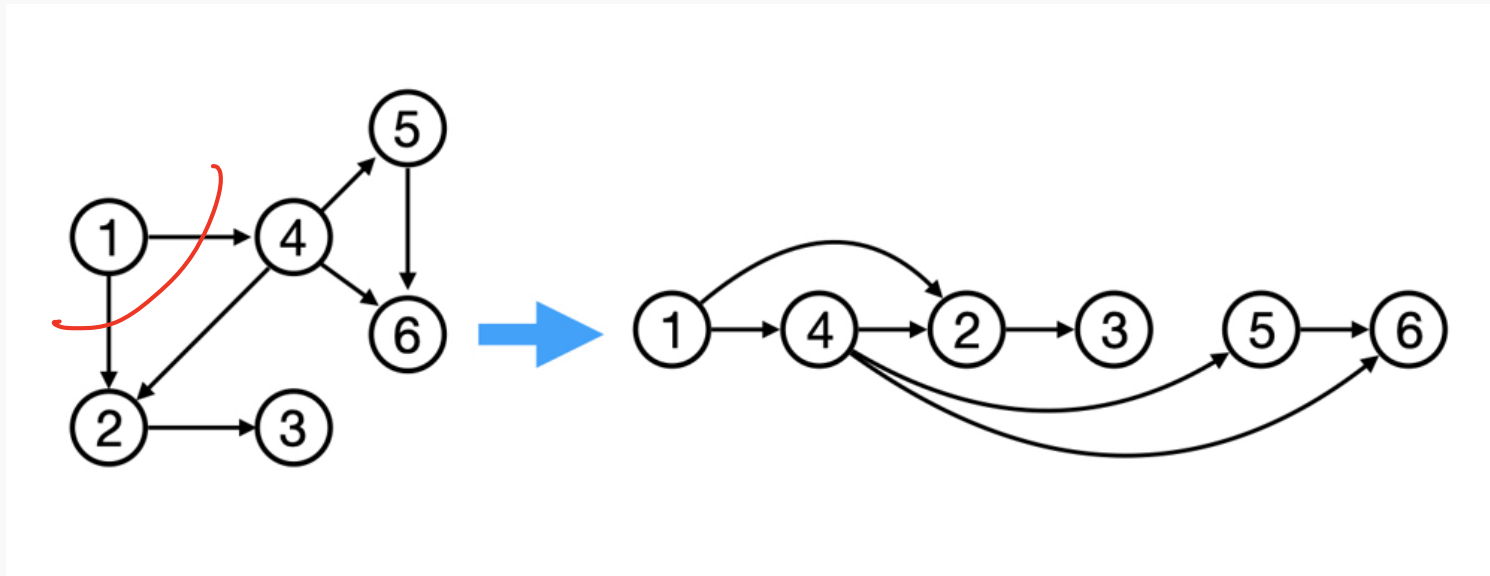


Topological Sort

- Input: a DAG $G = (V, E)$
- Output: an ordering of nodes such that for each edge $u \rightarrow v$, u comes before v
- There can be many answers.

partial ordering (not unique!)

extension to total ordering?



Topological Sort Algorithms

(straightforward algorithm)

The optimal version

- Any node without an incoming edge can be the first element
- After deciding the first node, remove outgoing edges from it
- Repeat!

start
from the
0-degree
vertices

- Time complexity: $\mathcal{O}(n^2 + m)$

- Too slow...

try to
make them
0-degree
to have

- Precompute the number of incoming edges $\deg(v)$ for each node v
- Put all nodes v with $\deg(v) = 0$ into a queue Q
- Repeat until Q becomes empty:
 - Take v from Q
 - For each edge $v \rightarrow u$:
 - Decrement $\deg(u)$ (essentially removing the edge $v \rightarrow u$)
 - If $\deg(u) = 0$, push u to Q

Time complexity: $\Theta(n + m)$

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

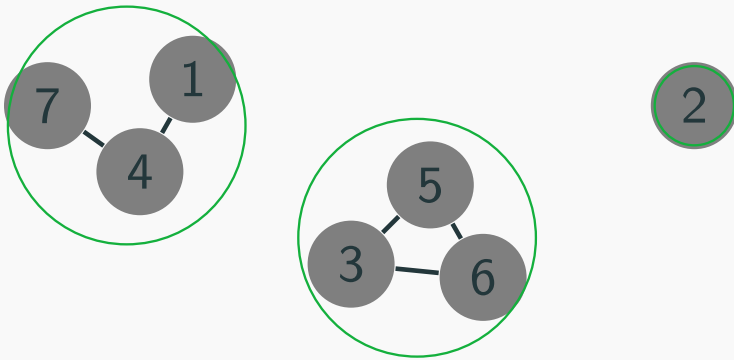
Shortest path(s)



Connected components (straight forward algorithm)

Start from a vertex and
go as far as you can

- Find the connected components using depth-first search.
- Pick a vertex and do a depth-first search from that vertex.
- All vertices reachable from the starting vertex are in the same component.
- Repeat this process until you have all the components.
- Time complexity is $\mathcal{O}(|V| + |E|)$.



```
vector<int> adj[1000];  
vector<int> component(1000, -1);
```

```
void find_component(int cur_comp, int u) {  
    if (component[u] != -1) {  
        return;  
    }  
  
    component[u] = cur_comp;  
  
    for (int i = 0; i < adj[u].size(); i++) {  
        int v = adj[u][i];  
        find_component(cur_comp, v);  
    }  
}
```

```
int components = 0;  
for (int u = 0; u < n; u++) {  
    if (component[u] == -1) {  
        find_component(components, u);  
        components++;  
    }  
}
```

Strongly Connected Components (SCC)

- Given a *directed* graph $G = (V, E)$
- A graph is **strongly connected** if all nodes are reachable from every single node in V
- Strongly connected components of G are maximal strongly connected subgraphs of G
- The following graph has 3 SCCs: $\{a, b, e\}$, $\{c, d, h\}$, $\{f, g\}$

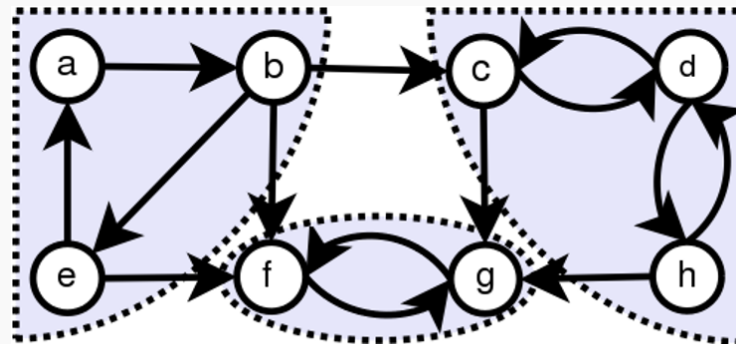


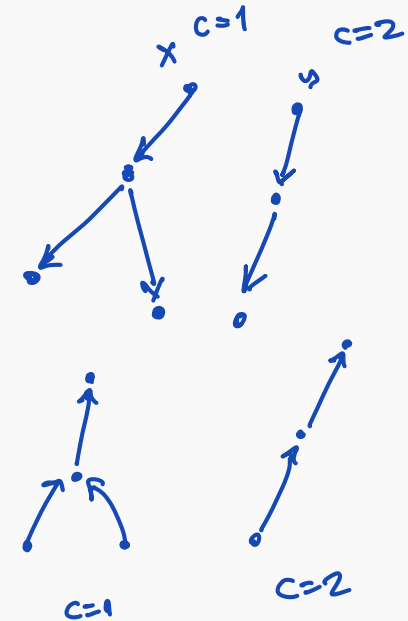
Figure from Wikipedia

Kosaraju's Algorithm [1978 unpublished]

[maybe also Shamir 1981]

2 DFS

- Initialize a counter $c := 0$
- While not all nodes are labeled:
 - Choose an arbitrary unlabeled node v
 - Start DFS from v
 - Mark the current node x as visited
 - Recurse on all its unvisited neighbors
 - After the DFS calls are finished, increment c and set the label of x as c
- Reverse the direction of all the edges
- For node v with label $n, n - 1, \dots, 1$:
 - Find all reachable nodes from v and group them as an SCC



Kosaraju's Algorithm

Main property

A graph and its transpose have the same strongly connected components.

reverse the edges
of a directed graph

Complexity

$$O(|V| + |E|)$$

Asymptotically optimal.

Algorithm by [Tarjan: 1975]
1 Graph traversal

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

Shortest path(s)



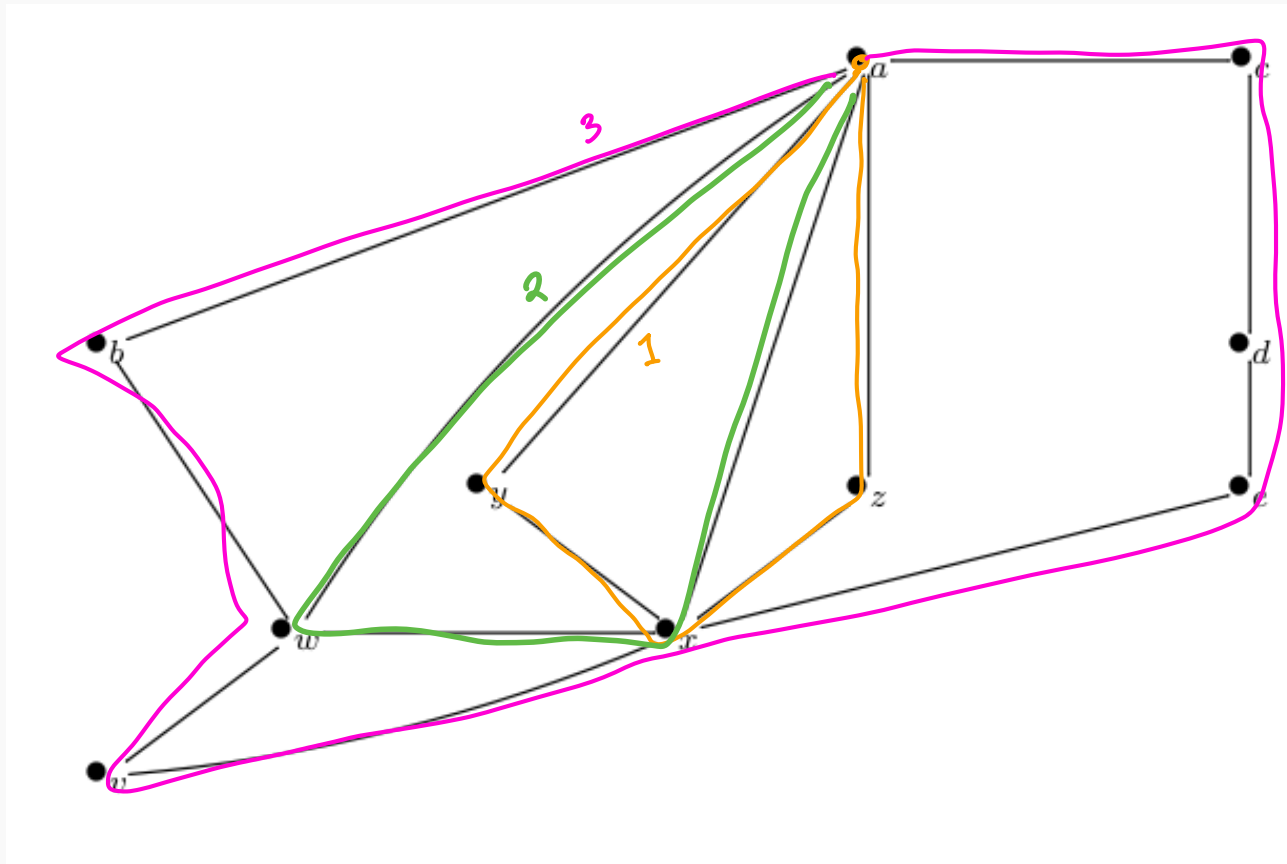
Eulerian Circuit

- Given an undirected graph G
- Find a sequence of nodes that visits every edge exactly once and goes back to the starting point
- Eulerian circuits exist if and only if
 - G is connected *and* each node has an even degree

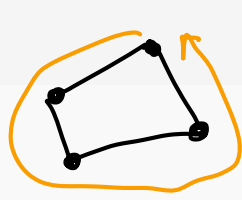
Constructive Proof of Existence

- Pick any node of G and walk randomly, but do not use the same edge twice.
- Each node is of even degree, so when you enter a node, there will be an unused edge you exit through.
 - Except at the starting point, at which you can get stuck.
- When you get stuck, you have a cycle.
 - Remove the cycle and repeat the process in each connected component.
 - Glue the cycles together to finish!

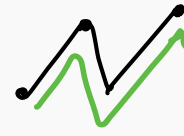
Find the Eulerian circuit (cycle)



Related Problems



circuit / cycle



Eulerian path

Eulerian path

Exists if and only if the graph is connected and the number of nodes with odd degree is 0 or 2.

Hamiltonian path/cycle

A path/cycle that visits every *node* in the graph exactly once.

Looks similar but very hard (still unsolved)! NP-hard

∃ exponential time algorithms

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

Shortest path(s)

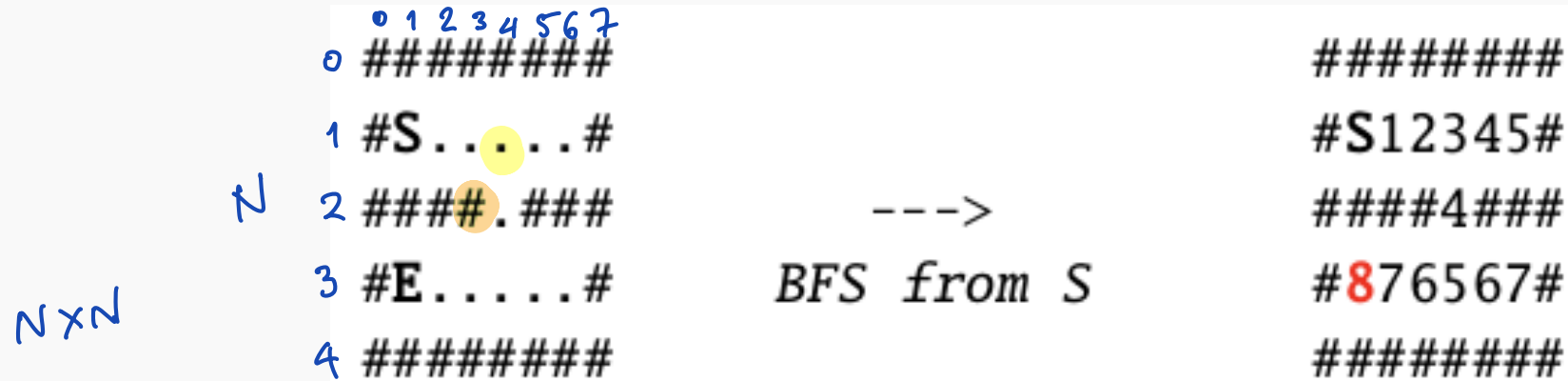


Escape the maze

the problem here is to find the graph

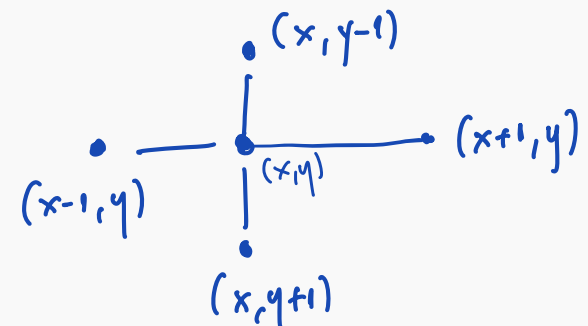
If we define a graph, then we can use BFS/DFS

- shortest distance in a maze

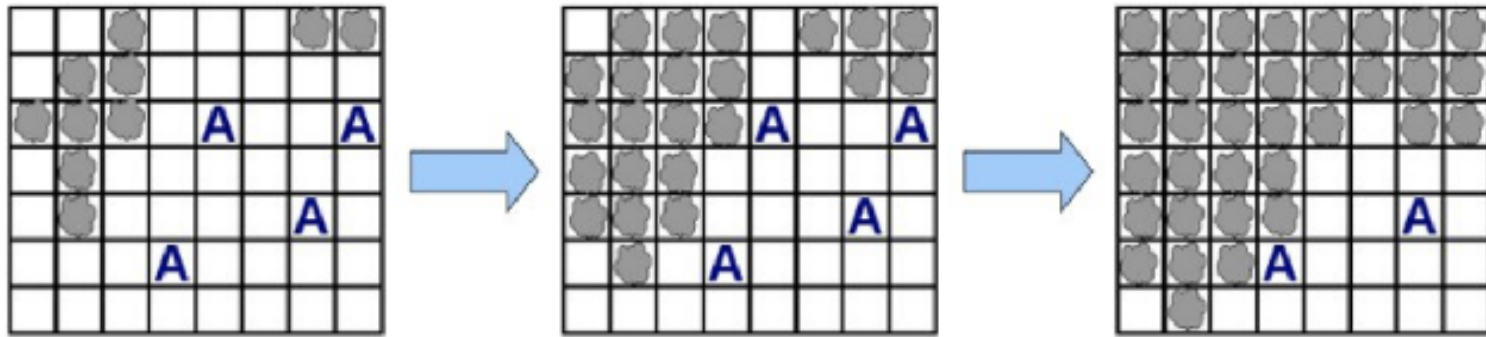


- Define the graph:

- You have a node for every position (x, y) .
- The degree of a node is at most 4.
- Do a BFS and compute distances (how?)
- Complexity $\mathcal{O}(\text{rows} \times \text{cols})$.



Can we land? [Moore 50s pixel map]



...#...	..1#1..	.21#12.	321#123	321#123
..##...	.1##1..	21##12.	21##123	21##123
.#####	-> 1#####1.	-> 1#####12	-> 1#####12	-> 1#####12
.....	11111..	111112.	1111123	1111123
##.....	##1....	##122..	##1223.	##12234

Hints

- Is a configuration valid (i.e., reachable)?
- You have sequence of moves and you want to find the smallest/longest distance of a valid (i.e., reachable) configuration.

If space matters, then we can do some practical optimizations

Contents

Introduction

Graph traversal

DFS tree and Bridges

Topological sort

(strongly) connected components

Eulerian Circuit

Other problems

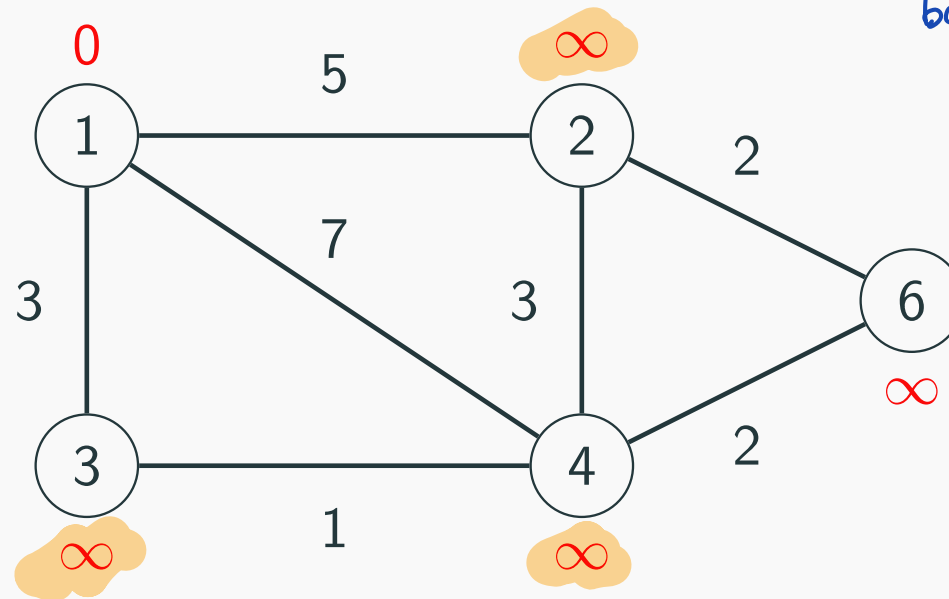
Shortest path(s)



Shortest paths (Belman–Ford)

all the paths from a source

Assume that there are no negative circles.



BFS
but keep track
of the edge weights
 $O(m+n)$